

Graph Mining Course Project Proposal

دوره: گراف کاوی

استاد: دکتر زینب مالکی

اطلاعات دانشجویان:

امیرحسین صادقی – آرین مدنی

شماره دانشجویی: ۴۰۰۱۱۶۴۳-۴۰۰۱۶۴۲۳

ایمیل:

Sadeghi.a@ec.iut.ac.ir

a.madani@ec.iut.ac.ir

Project Title

بررسی تعمیم‌پذیری مدل GraphSAGE از دیتاست Cora به دیتاست مشابه Citeseer

Abstract

در این پروژه قصد داریم عملکرد مدل GraphSAGE را در مسئله طبقه‌بندی گره‌ها روی دو دیتاست استاندارد شبکه‌های استنادی، یعنی Cora و Citeseer، بررسی و مقایسه کنیم. مدل GraphSAGE در مقاله‌ی اصلی معرفی خود عمدتاً روی دیتاست‌هایی مانند Cora، Pubmed و Reddit ارزیابی شده و اجرای صریح آن روی Citeseer در پیاده‌سازی‌های رسمی گزارش نشده است. هر دو دیتاست Cora و Citeseer از نظر نوع داده (مقالات علمی)، نوع یال‌ها (روابط استنادی)، وجود ویژگی‌های متنی برای گره‌ها و داشتن برچسب‌های موضوعی، ساختاری مشابه دارند؛ با این حال از نظر اندازه، تراکم و جزئیات شبکه متفاوت‌اند. ایده‌ی اصلی پروژه این است که همان معماری و تنظیمات GraphSAGE که روی Cora استفاده می‌شود را بدون تغییر اساسی روی Citeseer اعمال کنیم و تفاوت عملکرد را با توجه به تفاوت‌های ساختاری دو گراف تحلیل کنیم. خروجی نهایی شامل تحلیل ساختار گراف‌ها، مقایسه‌ی دقت و معیارهای ارزیابی و بحث در مورد میزان تعمیم‌پذیری GraphSAGE بین دو شبکه‌ی علمی خواهد بود.

Problem and Motivation

یکی از اهداف اصلی این پروژه، بررسی این سؤال است که آیا یک مدل گراف‌نورونی مانند GraphSAGE که روی یک شبکه‌ی استنادی علمی آموزش و ارزیابی شده، می‌تواند بدون تغییر اساسی روی یک شبکه‌ی استنادی مشابه دیگر نیز عملکرد خوبی داشته باشد یا نه. برای این منظور دو دیتاست کلاسیک Cora و Citeseer را انتخاب می‌کنیم که هم از

نظر نوع داده و هم از نظر ویژگی‌های ساختاری، شباهت زیادی به یکدیگر دارند. در هر دو دیتاست، گره‌ها نماینده‌ی مقالات علمی و یال‌ها نشان‌دهنده‌ی رابطه‌ی استناد بین آن‌ها هستند؛ علاوه بر این، برای هر گره یک بردار ویژگی متنی (بر اساس کلمات موجود در مقاله) و یک برچسب موضوعی (کلاس مقاله) در دسترس است. بنابراین هر دو گراف از یک نوع واحد هستند: شبکه‌ی استنادی با ویژگی و برچسب گره.

شباهت این دو شبکه فقط مفهومی نیست، بلکه در شاخص‌های کمی ساختار گراف نیز دیده می‌شود. میانگین درجه در Cora و Citeseer هر دو در حدود ۳ تا ۴ است؛ یعنی هر مقاله به طور متوسط به چند مقاله‌ی محدود متصل است و هر دو گراف نسبتاً تنک (sparse) هستند. ضریب خوشه‌بندی متوسط در هر دو شبکه در بازه‌ای تقریباً بین ۰/۱ تا ۰/۳ قرار می‌گیرد که نشان می‌دهد همسایه‌های یک گره تمایل قابل توجهی به تشکیل مثلث‌ها و خوشه‌های محلی دارند و ساختار گراف‌ها از یک گراف تصادفی ساده غنی‌تر است. همچنین طول میانگین کوتاه‌ترین مسیر در هر دو شبکه در مقیاس چند گام (تقریباً ۴ تا ۶ گام) است؛ به این معنا که بین هر دو مقاله معمولاً می‌توان از طریق تعداد کمی واسطه مسیر پیدا کرد و هر دو گراف خاصیت «دنای کوچک» (small-world) را تا حدی نمایش می‌دهند. علاوه بر این، هر دو شبکه یک کامپوننت عظیم اصلی دارند که بیشتر گره‌ها داخل آن قرار می‌گیرند، و توزیع درجه‌ی آن‌ها دم‌دراز (وجود چند گره‌ی با درجه‌ی بالا و تعداد زیادی گره با درجه‌ی کم) است.

ترکیب این ویژگی‌ها نشان می‌دهد که Cora و Citeseer دو شبکه‌ی «مشابه ولی نه یکسان» هستند: نوع داده، ساختار کلی، محدوده‌ی درجه و رفتار فاصله‌ای آن‌ها به هم نزدیک است، اما اختلافاتی در اندازه، تراکم یال‌ها و توزیع دقیق درجات وجود دارد. این وضعیت آن‌ها را برای مطالعه‌ی تعمیم‌پذیری مدل GraphSAGE بسیار مناسب می‌کند. در این پروژه قصد داریم همان معماری و تنظیمات GraphSAGE را ابتدا روی Cora و سپس بدون تغییر بنیادی روی Citeseer اجرا کنیم و مشاهده کنیم که تفاوت‌های ساختاری نسبتاً کوچک بین این دو گراف تا چه حد می‌تواند روی عملکرد مدل در مسئله‌ی طبقه‌بندی گره‌ها (از نظر Accuracy و معیارهای F1) اثر بگذارد. این تحلیل هم از نظر تئوری (رابطه‌ی بین ساختار گراف و عملکرد مدل) و هم از نظر عملی (انتقال یک مدل آموزش‌دیده بین دو شبکه‌ی علمی مشابه) اهمیت دارد و به درک بهتر رفتار مدل‌های گراف‌نورونی کمک می‌کند.

Objectives

Objective 1: تحلیل و مقایسه‌ی ویژگی‌های ساختاری دو شبکه Cora و Citeseer با استفاده از معیارهایی مانند میانگین درجه، توزیع درجه، ضریب خوشه‌بندی، نسبت گره‌ها در بزرگ‌ترین کامپوننت و طول میانگین کوتاه‌ترین مسیر، و مستندسازی شباهت‌ها و تفاوت‌ها.

Objective 2: پیاده‌سازی و اجرای مدل GraphSAGE روی دیتاست Cora با استفاده از یک پیاده‌سازی موجود (مانند PyTorch Geometric یا کد رسمی GraphSAGE) و بازتولید تقریبی نتایج طبقه‌بندی گره‌ها (Accuracy و F1) روی این دیتاست.

Objective 3: آماده‌سازی دیتاست Citeseer در قالب سازگار با پیاده‌سازی GraphSAGE (شامل ماتریس ویژگی‌ها، ماتریس یال‌ها و تقسیم‌بندی train/validation/test) و اجرای همان معماری و تنظیمات مدل روی این دیتاست، بدون تغییر بنیادی در ساختار مدل.

Objective 4: مقایسه‌ی دقیق عملکرد GraphSAGE روی Cora و Citeseer با استفاده از معیارهای Accuracy، Micro-F1 و Macro-F1، و تحلیل ارتباط تفاوت‌های ساختاری دو گراف با تفاوت‌های مشاهده‌شده در نتایج مدل، همراه با بحث در مورد محدودیت‌ها و پیشنهادهای بهبود.

Related Work

مدل GraphSAGE توسط Hamilton و همکاران (۲۰۱۷) به‌عنوان یک روش یادگیری نمایشی القایی روی گراف‌ها ارائه شد. ایده‌ی اصلی این مدل این است که به جای یادگیری یک embedding جداگانه برای هر گره، یک تابع تجمیع (Aggregator) روی همسایه‌ها یاد گرفته شود تا بتواند روی گره‌های جدید و حتی گراف‌های جدید نیز اعمال شود. در مقاله‌ی اصلی، GraphSAGE روی دیتاست‌های Cora، Pubmed، Reddit و چند شبکه‌ی بزرگ دیگر ارزیابی شده و نشان داده شده که می‌تواند نسبت به روش‌های کلاسیک و برخی GNN‌های قبلی عملکرد بهتری داشته باشد.

از سوی دیگر، مدل GCN که توسط Kipf و Welling معرفی شده، یکی از اولین مدل‌های مشهور GNN برای طبقه‌بندی نیمه‌نظارتی گره‌هاست و در بسیاری از کارها روی Cora و Citeseer به‌عنوان baseline استفاده شده است. اکثر پیاده‌سازی‌های آموزشی موجود در GitHub نیز GCN را روی هر دو دیتاست اجرا کرده‌اند، در حالی که GraphSAGE معمولاً روی Cora و Pubmed تست می‌شود و مثال مستقیم برای Citeseer کمتر دیده می‌شود. پروژه‌ی ما روی همین فاصله تمرکز می‌کند: استفاده از GraphSAGE، که در ادبیات روی Cora خوب کار کرده، روی دیتاست مشابهی به نام Citeseer و بررسی این که آیا نتایج آن هم‌راستا با انتظار ما از یک مدل القایی است یا نه.

Proposed Methodology

Dataset(s)

در این پروژه از دو دیتاست استاندارد شبکه‌های استنادی استفاده می‌کنیم:

- Cora: شامل حدود ۲۷۰۸ گره (مقاله) و ۵۴۲۹ یال استنادی است. هر گره یک بردار ویژگی دودویی یا عددی بر اساس حضور/عدم حضور کلمات در متن مقاله دارد و به یکی از ۷ کلاس موضوعی (مثلاً یادگیری ماشین، شبکه‌های عصبی و ...) نسبت داده شده است.
- Citeseer: شامل حدود ۳۳۲۷ گره و ۴۷۳۲ یال استنادی است. این گراف نیز برای هر مقاله یک بردار ویژگی متنی و یک برچسب موضوعی در ۶ کلاس مختلف دارد.

در مرحله‌ی پیش‌پردازش، یال‌های گراف‌ها را (در صورت نیاز) از حالت جهت‌دار به بدون‌جهت تبدیل می‌کنیم تا فرآیند انتشار اطلاعات در GNN ساده‌تر شود. سپس داده‌ها را به قالب مورد نیاز کتابخانه‌ی PyTorch Geometric تبدیل می‌کنیم: ماتریس ویژگی X با ابعاد [تعداد گره $\times$ تعداد ویژگی]، ماتریس یال‌ها به صورت `edge_index` با فرمت COO،

و بردار برجسبها y. یک تقسیم‌بندی استاندارد train/validation/test (مثلاً تقسیم‌های مرسوم برای Cora/Citeseer) تعریف می‌کنیم تا نتایج بین دو دیتاست به شکل منصفانه قابل مقایسه باشد.

Techniques and Algorithms

- GraphSAGE: مدل اصلی پروژه است. در ساده‌ترین حالت، از تجمیع‌کننده‌ی Mean Aggregator برای ترکیب ویژگی‌های گره و همسایه‌هایش استفاده می‌کنیم. معماری پایه می‌تواند شامل دو لایه GraphSAGE با اندازه‌ی لایه‌ی میانی ثابت (مثلاً ۶۴ نورون) باشد، به همراه تابع فعال‌سازی ReLU، Dropout برای جلوگیری از بیش‌برازش و یک لایه‌ی خطی نهایی برای نگاشت به تعداد کلاس‌ها. ابتدا این مدل را روی Cora آموزش می‌دهیم و هاپیرپارامترها (نرخ یادگیری، تعداد epochs، نسبت Dropout و ...) را در محدوده‌ی معقول تنظیم می‌کنیم.

- GCN (Baseline اختیاری): برای داشتن یک خط مبنا، در صورت امکان یک مدل GCN ساده نیز روی هر دو دیتاست اجرا می‌کنیم تا تفاوت GraphSAGE با یک GNN کلاسیک را بهتر تحلیل کنیم.

- ابزارها: از Python و کتابخانه‌های PyTorch و PyTorch Geometric برای پیاده‌سازی مدل‌ها، و در صورت نیاز از NetworkX برای محاسبه‌ی برخی ویژگی‌های ساختاری گراف (مانند ضریب خوشه‌بندی و میانگین طول کوتاه‌ترین مسیر) استفاده می‌کنیم. خروجی‌ها شامل مقادیر عددی معیارها و نمودارهای ساده‌ی آموزش/اعتبارسنجی خواهند بود.

Evaluation Plan

ارزیابی عملکرد مدل عمدتاً بر اساس مسئله‌ی طبقه‌بندی گره‌ها انجام می‌شود. معیارهای زیر را برای هر دو دیتاست گزارش می‌کنیم:

- Accuracy: روی مجموعه‌ی تست: درصد گره‌هایی که برجسب آن‌ها به‌درستی پیش‌بینی شده است.

- Micro-F1 و Macro-F1: برای بررسی بهتر عملکرد وقتی توزیع کلاس‌ها نامتوازن باشد و اهمیت همه‌ی کلاس‌ها را یکسان در نظر بگیریم.

- در صورت امکان، Confusion Matrix برای هر دیتاست جهت تحلیل کلاس‌هایی که مدل در آن‌ها بیشترین خطا را دارد.

برای مقایسه، ابتدا نتایج GraphSAGE روی Cora را با نتایج گزارش‌شده در مقاله یا پیاده‌سازی‌های مرجع مقایسه می‌کنیم تا مطمئن شویم پیاده‌سازی ما رفتار معقولی دارد. سپس همان معماری و هاپیرپارامترها را روی Citeseer اعمال کرده و نتایج را ثبت می‌کنیم. اگر Baseline GCN را نیز پیاده‌سازی کنیم، دقت و F1 آن را هم روی هر دو دیتاست گزارش می‌کنیم تا مشخص شود که GraphSAGE نسبت به یک مدل مرسوم چه تغییری در عملکرد ایجاد می‌کند. در نهایت، تفاوت عملکرد بین دو دیتاست را در کنار تفاوت‌های ساختاری گراف‌ها (مثلاً میانگین درجه، تعداد کامپوننت‌ها و ...) بررسی می‌کنیم.

Challenges and Resources

چند چالش اصلی پیش‌بینی می‌شود. نخست، آماده‌سازی دقیق دیتاست Citeseer در قالب سازگار با پیاده‌سازی GraphSAGE است؛ ممکن است لازم باشد فرمت داده، نحوه ساخت edge_index یا تقسیم‌بندی train/val/test را با دقت تنظیم کنیم تا نتایج پایدار و قابل اعتماد باشند. دوم، تنظیم هاپرپارامترهای مدل است؛ اگر تنظیمات بیش از حد برای Cora بهینه شده باشند، ممکن است روی Citeseer عملکرد ضعیف‌تری بدهند و مقایسه را سخت کنند. برای مدیریت این چالش‌ها، از مستندات PyTorch Geometric، ریپازیتوری‌های GitHub مربوط به GraphSAGE/GCN و نتایج منتشر شده برای Cora استفاده خواهیم کرد و سعی می‌کنیم تنظیمات را تا حد امکان منصفانه و ثابت نگه داریم.

References

- Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive Representation Learning on Large Graphs. Advances in Neural Information Processing Systems (NeurIPS).
- .GitHub – williamleif/graphsage-simple: Implementation of GraphSAGE
- PyTorch Geometric Documentation – <https://pytorch-geometric.readthedocs.io>